Fill-triggered journals, notifications and journal status

This manual uses the offline smoke, not account data or live orders. The PNG shows selected fields from the actual stdout of `python scripts/smoke_fill_journal.py`. That script drives an in-process supervisor and then real CLI processes against a temporary SQLite file, with exchange reads served from a local fills file and sockets blocked.

Before you start

- Supervisors (`supervisor run`) and the worker (`market collect`) use the **same** absolute `--db` path.
- One `account` job per account: Hyperliquid every 300 s and KIS every 3,600 s. Example job JSON: `{"kind":"account","venue":"hyperliquid","account":"ACTUAL_SUBACCOUNT","start_ms":VERIFIED_START}`.
- `market collect --poll-seconds 30` is running. Installing it as a host service is a separately approved cutover step; nothing here starts it.

A supervised fill (scenario S1)

When a live supervisor observes a fill, it commits its own state and then records one sync request. On its next pass, `market collect` syncs that account from 15 minutes before the supervisor's previous observation and writes the journal. It then queues one `journal_batch` notification. Hermes reads and acknowledges it:

```
python -m kis_hl.cli supervisor notifications --venue hyperliquid
python -m kis_hl.cli supervisor notification-result --venue hyperliquid \
    --notification-id ID --result delivered
```

Each line shows the instrument, change type, signed quantity, price, KST time and source precision. A cycle-closing line shows realized PnL only when the cycle is finalized; otherwise it says `PnL pending (reasons)`. Routine syncs leave HL cash coverage and KIS trade coverage unverified, so pending is the normal result until coverage is certified.

```
$ python scripts/smoke_fill_journal.py   # selected stdout fields

# supervised fill -> journal_batch (Hermes acknowledged: delivered=True)
JOURNAL hyperliquid: 1 new entry
- hl:BTC entry +1 @ 100 USDC, 2026-10-14 19:41:50 KST

# manual exchange-app close -> fallback account job
JOURNAL hyperliquid: 1 new entry
- hl:BTC close -1 @ 100 USDC, 2026-10-14 19:41:52 KST; PnL pending (cash_coverage_unverified,
    trade_coverage_unverified)

# repeat sync -> notification_id: null

# KIS overseas DAY statement -> date and precision, no time of day
JOURNAL kis: 1 new entry
- kis:SPY unclassified +1 @ 500 USD, 2026-10-14 KST (DAY precision)

# failed exchange read -> backoff (data journal-status: sync.request)
attempts=1 last_error=TimeoutError next_attempt_ms-last_attempt_ms=30000

# data journal-status --enqueue-summary -> journal_daily
DAILY JOURNAL 2026-10-14 (hyperliquid): no open positions; 2 journal entries today; closed
    cycles 1 (PENDING 1); statistics pending (1 cycle not finalized); pending:
    cash_coverage_unverified 1, trade_coverage_unverified 1; sync request pending, 1 failed
    attempt (TimeoutError)
```

Manual trades and repeats (scenario S2)

A trade placed in the exchange app has no supervisor. The account job picks it up within its interval: 5 minutes for HL, 60 minutes for KIS. Syncing the same range again (for example `data sync` for an audit) adds no journal and no notification. In the smoke, the repeat sync returned `notification_id: null`.

KIS DAY precision (scenario S4)

KIS statements (domestic and overseas) are daily and cumulative. Their lines show the KST date and `(DAY precision)`, never an invented time of day. The capture above includes an overseas DAY statement synced through `data sync --venue kis`.

When a sync fails (scenario S3)

A failed exchange read never touches protection or orders. The request stays pending and records `attempts` and `last_error` (the exception type only), then retries after 30 s, 60 s and so on, up to 15 minutes, until a success clears it. Check it with:

```
python -m kis_hl.cli data journal-status
```

The `sync` part shows the request, the account jobs, the last failed collection and the worker heartbeat (`worker_last_seen_ms`). If requests stay pending and the heartbeat is old, the worker is not running.

Daily summary (scenario S5)

`data journal-status` is read-only; the database is byte-for-byte unchanged afterwards. After the US close, Hermes runs:

```
python -m kis_hl.cli data journal-status --enqueue-summary
```

This queues one `journal_daily` notification per account and KST date. Running it again on the same date queues nothing. The summary lists open positions, today's entries, closed cycles, and the nine statistics: finalized cycles only, otherwise "statistics pending" with the count. It also lists pending reasons and sync health. Delivery is at least once, so a crash between sending and `notification-result` can repeat a message.

See the sync policy.